POWER BI

From Zero to Expert

A step-by-step course for beginners who want professional skills

Prepared by Jean Baptiste Habanabakize, Data Analytics Specialist

No experience needed. Build your first report in sixty minutes.

Every step explains *why* it matters, what you should *see* on screen, and what to do when you *don't*.

Jean Baptiste Habanabakize

Master in Big Data Analytics (Data Science), 2026

Contents

Welcome, and How to Learn From This Course 4

1 What Is Power BI? 6
- 1.1 The three products . . . 6
- 1.2 The five stages . . . 6

2 Your First Report in Fifteen Minutes 8
- 2.1 Install Power BI Desktop . . . 8
- 2.2 A tour of the interface . . . 8
- 2.3 Load the sales data . . . 9
- 2.4 Make your first visuals . . . 10
- 2.5 Save your work . . . 10
- 2.6 Turn off automatic date tables . . . 11

3 Clean and Shape Data with Power Query 12
- 3.1 The Power Query editor . . . 12
- 3.2 Rename queries . . . 13
- 3.3 Parameterise the file path . . . 13
- 3.4 Set data types . . . 14
- 3.5 Add a conditional column . . . 15
- 3.6 Column from Examples . . . 15
- 3.7 Handling time zones . . . 16
- 3.8 Append the 2023 data . . . 16
- 3.9 Merge queries (joins) . . . 17
- 3.10 Unpivot . . . 17
- 3.11 Error handling . . . 18
- 3.12 Query folding . . . 18

4 Model the Data 20
- 4.1 The star schema . . . 20
- 4.2 Create the relationships . . . 21
- 4.3 Build and mark the Date table . . . 22
- 4.4 Role-playing dates . . . 23
- 4.5 Many-to-many relationships . . . 23
- 4.6 Hide, organise, and categorise . . . 23

5 DAX: Calculating Answers 25
- 5.1 The Two Contexts: How DAX Thinks . . . 25
- 5.2 Step 1: The Base Measures . . . 25
- 5.3 Step 2: Filtering with CALCULATE . . . 26
- 5.4 Step 3: Variables (VAR) and RETURN . . . 28

5.5 Step 4: Advanced Analysis (Top Performers) . . . 28
5.6 Step 5: Time Intelligence . . . 29
5.7 Practice: Test Your Knowledge . . . 29

6 Visualise and Analyse **31**
6.1 Choose the right chart . . . 31
6.2 Build the Overview page . . . 32
6.3 Add a slicer and edit interactions . . . 32
6.4 Drillthrough . . . 32
6.5 Tooltips and bookmarks . . . 33
6.6 Design rules for readable reports . . . 33

7 Publish, Secure, and Refresh **35**
7.1 Workspaces, apps, and roles . . . 35
7.2 Publish the report . . . 36
7.3 Refresh and gateways . . . 36
7.4 Static row-level security . . . 36
7.5 Dynamic row-level security . . . 37

8 Performance and Tooling **39**
8.1 Four cost centres . . . 39
8.2 Cardinality . . . 39
8.3 Storage modes . . . 40

9 Building an Organizational Dashboard **42**
9.1 The Golden Rules of Dashboard Design . . . 42
9.2 Building the Executive Summary Page . . . 43
9.3 Building the Operational Detail Page . . . 43
9.4 Adding Professional Polish . . . 43
9.4.1 Navigation Buttons . . . 44
9.4.2 Custom Tooltips . . . 44
9.5 Practice: Apply What You Have Learned . . . 44

10 Enterprise Practice: Deployment and Governance **45**
10.1 Deployment pipelines . . . 45
10.2 Power BI project files and version control . . . 45
10.3 Certification and labels . . . 45

11 NGO and Monitoring & Evaluation Patterns **47**
11.1 Indicators, targets, and actuals . . . 47
11.2 Disaggregation . . . 48
11.3 Cleaning KoboToolbox and CommCare exports . . . 48

12 Exam Preparation and PL-300 Mapping **50**
12.1 Mapping to the exam . . . 50
12.2 Practice questions . . . 50
12.3 Answer key with reasoning . . . 52
12.4 Scenario questions . . . 53

13 Capstone Project **54**
13.1 Brief . . . 54
13.2 Deliverables . . . 54
13.3 Marking rubric . . . 55

A Data File Reference 56

B Cheat Sheets 57

C Glossary 59

D About the Author and Disclaimer 60

Welcome, and How to Learn From This Course

Welcome. I wrote this course the way I would teach it in a classroom: one idea at a time, with the reason behind every click. Power BI can look intimidating on the first day, with its menus, panes, and languages. My promise is that by the end you will understand not only *how* to build a report, but *why* each step is designed the way it is. That understanding is what lets you pass an exam and, more importantly, solve a problem that no tutorial has shown you.

How each part is organised

Every part follows the same rhythm, so you always know what is coming.

1. **Goals** tell you what you should be able to do when the part is finished.
2. **Labs** give you the clicks, in numbered steps. Each step names the window you should be looking at.
3. **Why this matters** explains the reasoning behind a step, so you can adapt it when your data is different.
4. **Check yourself** tells you what result you should see. If your screen differs, stop and fix it before you move on.
5. **Watch out** warns you about mistakes that almost every beginner makes.
6. **Go deeper** gives optional theory. You can skip it on your first pass and come back later.
7. **Practice** asks you to work without the steps, followed by worked **Solutions**.

> **A word from your lecturer**
>
> Please do the practice questions before you look at the solutions. Struggling for ten minutes teaches you more than reading a solution in ten seconds. If you get stuck, reread the part's "Why this matters" notes, because most answers follow from them.

How screen text is written

When you see ***bold italic text***, it is a button, menu, tab, or field that you click or select on the screen. When you see `typewriter text`, you type it exactly as written. Power BI menus are sometimes translated or renamed by Microsoft, so if a name on your screen differs slightly, look for the closest match.

Expected values by stage

The checkpoints in this course come from the sample dataset. The value you should see depends on how far you have got, so use this table to find the right number for your stage.

Where you are	What you should see	Part
Sales.csv loaded, no LineAmount column yet	Quantity total 40; distinct OrderID 12	2
LineAmount added, before the 2023 file is appended	Total Sales 2,065	3
After appending Sales_2023_Extension	Total Sales 2,805 (2,065 plus 740 from 2023)	3

Watch out

The source notes do not agree on how many rows `Sales.csv` contains. One part says the preview shows 12 rows and another says 15. Count the rows in your own file first. If your count differs, trust your file, check your data types and steps, and then carry on. Your file is the truth; the checkpoints are guides.

What you need

- Windows 10 or 11. Power BI Desktop does not run natively on a Mac, so use a Windows PC or a Windows virtual machine.
- A free Microsoft account (any email address will do).
- About fifteen minutes for Part 2, around four hours for the whole course, and extra time for the practice questions.

The data files

Create the folder `C:/PowerBI_Course/data` and place four files inside it: `Sales.csv`, `Customers.csv`, `Products.csv`, and `Sales_2023_Extension.csv`. Appendix A lists the columns each file must contain. **Do not open the CSV files in Excel before loading them.** Excel can change the text encoding and the date formats silently, and then Power BI reads different values from the ones you see.

One project throughout

Every part adds to the same file, `DataHills.pbix`. By the end you will have a star-schema model, a set of tested DAX measures, drillthrough pages and bookmarks, static and dynamic row-level security, a scheduled refresh, and a published app. The capstone in Part 13 then asks you to repeat the whole workflow on a new dataset, which is the real test of whether you have learned it.

Part 1

What Is Power BI?

By the end of this part you will be able to

- Explain what Power BI does in a single sentence.
- Name the three Power BI products and say what each one is used for.
- Name the five stages that every report passes through.

Let us start with the problem Power BI solves. Imagine a manager who asks, "What were our sales in Kenya last quarter?" Before tools like this, somebody had to request a data extract, wait for it, and then build a spreadsheet by hand. Power BI connects directly to the data, does the calculation, and gives the manager an interactive page where they can click Kenya and select the quarter themselves.

1.1 The three products

Power BI is not one program. It is three, and the exam will ask you which one does which job.

Product	Where it runs	What you do with it
Power BI Desktop	Your Windows computer	Connect to data, clean it, model it, write calculations, and design the report. The work is saved in a `.pbix` file.
Power BI Service	A web browser, in the cloud	Publish reports, share them, schedule data refreshes, manage security, and monitor usage.
Power BI Mobile	A phone or tablet	Read and interact with reports that someone has already published.

A word from your lecturer

A simple way to remember it: Desktop is where you *build*, Service is where you *share*, and Mobile is where you *read on the go*. If an exam question describes a task, ask yourself which of those three verbs it matches.

1.2 The five stages

Every Power BI report, from a small classroom exercise to a bank's risk dashboard, passes through the same five stages. This course follows them in order, so the stages are your roadmap.

1. **Connect** to the data, from a file, a database, or the cloud.
2. **Transform** the data with Power Query: clean it, set the types, and reshape it.
3. **Model** the data by defining how the tables relate to each other.
4. **Calculate** with DAX, writing the measures that answer business questions.
5. **Visualise** the results, then **publish** and **secure** the report.

Watch out

It is tempting to skip stages two and three because they feel slow and dull. Do not. A model built on untyped, unmodelled data will work until someone asks a question it cannot answer. At that point it returns a wrong number, and it gives you no error message to warn you. Clean data and clean relationships are the foundation that everything else depends on.

Go deeper

Behind every visual sits an in-memory engine called **VertiPaq**. It stores each column separately and compresses it. A column with few distinct values, such as Country or Category, compresses extremely well. A column with millions of unique values, such as a timestamp to the second, is expensive. This means that model size depends on the number of *distinct values per column*, not on the number of rows. Part 8 shows you how to use this knowledge to make reports faster.

Practice

1. Which product would you use to schedule a nightly data refresh, and why?
2. Put these tasks in the correct order: Visualise, Connect, Model, Transform, Calculate.
3. In two or three sentences, explain how a wrong data type in Stage 2 can produce a wrong number in Stage 5.

Solutions

1. Power BI Service. Refresh is configured in the Service, because the Service runs on a schedule without anyone's computer being switched on.
2. Connect, Transform, Model, Calculate, Visualise.
3. A number stored as text cannot be added up, and a key stored as text may not match a key stored as a number. Either problem can leave rows out of a total. The visual then shows a smaller number than it should, and nothing in the report warns you.

Part 2

Your First Report in Fifteen Minutes

By the end of this part you will be able to

- Install Power BI Desktop and identify its five main areas.
- Load a CSV file and build your first cards from it.
- Save a `.pbix` file and turn off automatic date tables.

Today you will build something that works. That feeling matters, because it turns a new tool into a familiar one. We will keep this part short and concrete.

2.1 Install Power BI Desktop

Install and open Power BI Desktop

Step 1. [Windows] Open the ***Microsoft Store*** from the Start menu.
Step 2. [Microsoft Store] Search for ***Power BI Desktop***.
Step 3. [Microsoft Store] Click ***Get***, then ***Install***, and wait for the installation to finish.
Step 4. [Windows] Open ***Power BI Desktop***.
Step 5. [Main Window] Sign in with a Microsoft account when prompted.
Step 6. [Main Window] On the start screen, click ***Blank report***.

Why this matters: Desktop is your workshop. Everything you design lives in a `.pbix` file on your own computer. The Service is for sharing that work with other people, and we reach it later in the course.

Check yourself

You should see an empty white canvas. A tall ***Visualizations*** panel sits on the right, and a ***Data*** panel sits beside it.

If something goes wrong: If you see a web page instead of the canvas, you have probably opened the browser version of Power BI Service. Close that tab and open Power BI Desktop from the Start menu.

2.2 A tour of the interface

Before you build anything, learn where things live. The drawing below is a simplified picture of the Desktop window. Your screen may differ slightly in layout, but the five areas are always there.

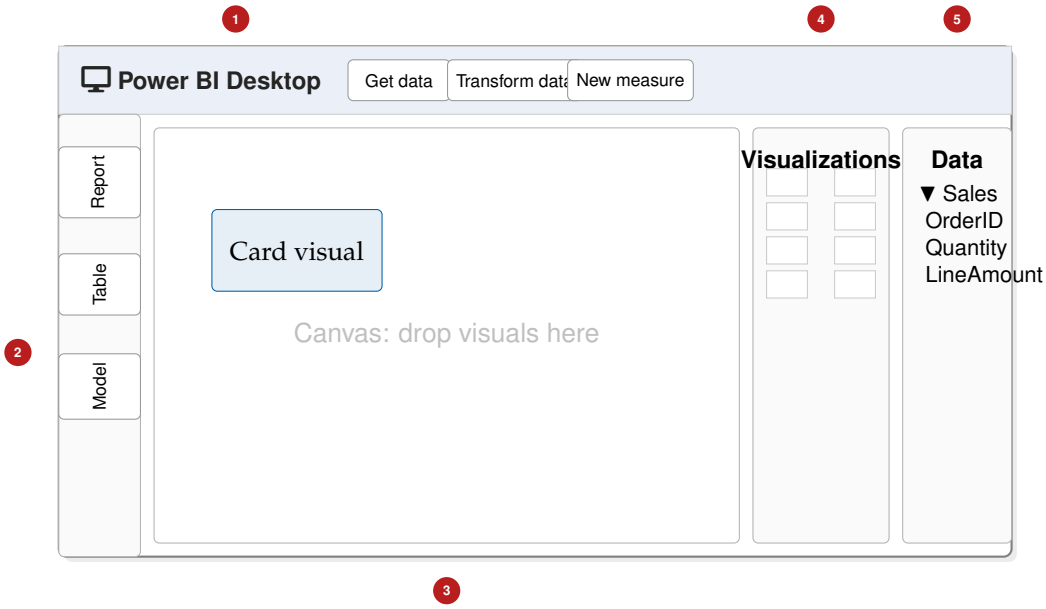


Figure 2.1: The Power BI Desktop window (schematic). The red numbers match the list below.

1. **The ribbon** across the top is the menu bar. ***Get data*** brings in tables, and ***New measure*** starts a DAX formula.
2. **The left edge tabs** switch between three views: ***Report*** (your pages), ***Table*** (the raw rows), and ***Model*** (the relationships between tables).
3. **The canvas** in the middle is the page where you place your visuals.
4. **The Visualizations pane** on the right offers chart types, and the ***Fields*** and ***Format*** settings for whichever visual you have selected.
5. **The Data pane** on the far right lists every table and column. You drag fields from here onto the canvas.

Check yourself

Point to all five areas on your own screen and name them out loud. If you can do this without looking at Figure 2.1, you are ready for the next section.

2.3 Load the sales data

Load Sales.csv

Step 1. [Main Window] In the ribbon, click ***Get data*** → ***Text/CSV***.

Step 2. [File Browser] Open `C:/PowerBI_Course/data/Sales.csv`.

Step 3. [Preview Window] Look at the preview before you do anything else. The first row should contain the headers `OrderID`, `OrderDate`, `CustomerKey`, `ProductKey`, `Quantity`, `UnitPrice`.

Step 4. [Preview Window] Click ***Load***.

*Why this matters: Clicking **Load** brings the data in exactly as it is. The button **Transform data** opens Power Query, the cleaning editor that you will meet in Part 3. For now we only want the data inside the report.*

Check yourself

The ***Data*** pane now shows a table called **Sales** with six columns.

*If something goes wrong: If the headers appeared as a data row, click **Transform data**, choose **Use First Row as Headers**, and then click **Close & Apply**.*

Watch out

Loading a file without looking at its preview is how most broken models begin. Take two seconds to read the first rows before you click ***Load***. It is the cheapest quality check you will ever do.

2.4 Make your first visuals

Here is a small detail that trips up many beginners. The column `LineAmount` does not exist in the raw file. We create it in Part 3. So for now we use columns that are already there.

Two cards from the raw file

Step 1. [Main Window] Expand **Sales** in the ***Data*** pane.

Step 2. [Main Window] Drag ***Quantity*** onto the empty canvas. The card shows a total.

Step 3. [Main Window] Drag ***OrderID*** onto the canvas next to it. Click the small arrow on the card, and change the aggregation to ***Count (Distinct)***.

Why this matters: When you drop a numeric column onto the canvas, Power BI aggregates it for you. A sum adds the values together. A distinct count tells you how many different identifiers there are. For order numbers, a distinct count is the right question, because a sum of order numbers means nothing at all.

Check yourself

The first card shows the Quantity total, which should be 40. The second shows the number of distinct orders, which should be 12. These are the checkpoints from the table at the start of the course.

*If something goes wrong: If OrderID shows a large number such as 12,006, the aggregation is still set to Sum. Change it to **Count (Distinct)**.*

2.5 Save your work

Save the file

Step 1. [Main Window] Click ***File*** → ***Save as***.

Step 2. [File Browser] Save the file in `C:/PowerBI_Course` with the name `DataHills.pbix`.

Why this matters: Saving early gives you a known-good state to go back to. Everybody breaks a model at some point, and a saved copy turns a disaster into a five-minute recovery.

2.6 Turn off automatic date tables

Disable Auto date/time

Step 1. [Main Window] Click ***File*** → ***Options and settings*** → ***Options***.

Step 2. [Options Window] Under ***Current File***, choose ***Data Load***, then untick ***Auto date/-time***.

Step 3. [Options Window] Click ***OK***.

Why this matters: With Auto date/time switched on, Power BI quietly creates a hidden date table for every date column in your file. The model grows, refreshes slow down, and time intelligence can behave in ways you did not expect. The professional approach is to build one explicit Date table and share it across the model. You will do exactly that in Part 4.

Watch out

This setting belongs to each individual `.pbix` file, not to Power BI as a whole. Switch it off in every new file you create.

What you have just built. A saved report with visuals built from loaded data. That is the core loop of Power BI: data goes in, and a visual comes out. Every later part adds one more layer on top of this loop.

Practice

1. Load `Customers.csv` without transforming it. Build a card that counts its rows. Which aggregation did you use, and why?
2. Build a card that shows the number of distinct `ProductKey` values in Sales. Why must you use a distinct count rather than a sum?
3. Turn off Auto date/time for your file, then save and close Power BI Desktop.

Solutions

1. Use ***Count*** on a column that has a value in every row, such as `CustomerKey`. When every row has a unique identifier, counting rows and counting identifiers give the same answer.
2. A sum of product keys adds up identifiers, which is meaningless. A distinct count tells you how many different products appear in the data.
3. Go to Options, then Data Load, untick Auto date/time, click OK, and save.

Part 3

Clean and Shape Data with Power Query

By the end of this part you will be able to

- Use the Power Query editor and read its three panels.
- Set data types, use a parameter for the file path, and write a calculated column in M.
- Add conditional columns, extract text from examples, and convert time zones.
- Append and merge queries, unpivot data, and handle errors deliberately.
- Explain query folding and why it affects refresh speed.

Power Query is where real projects are won or lost. Most of the errors that surface later, in a wrong total or a missing customer, began as a small cleaning decision made here. The good news is that Power Query records every step you take and replays those steps each time the data refreshes. You clean once, and the cleaning repeats automatically. That is far better than cleaning the same file by hand in Excel every month.

3.1 The Power Query editor

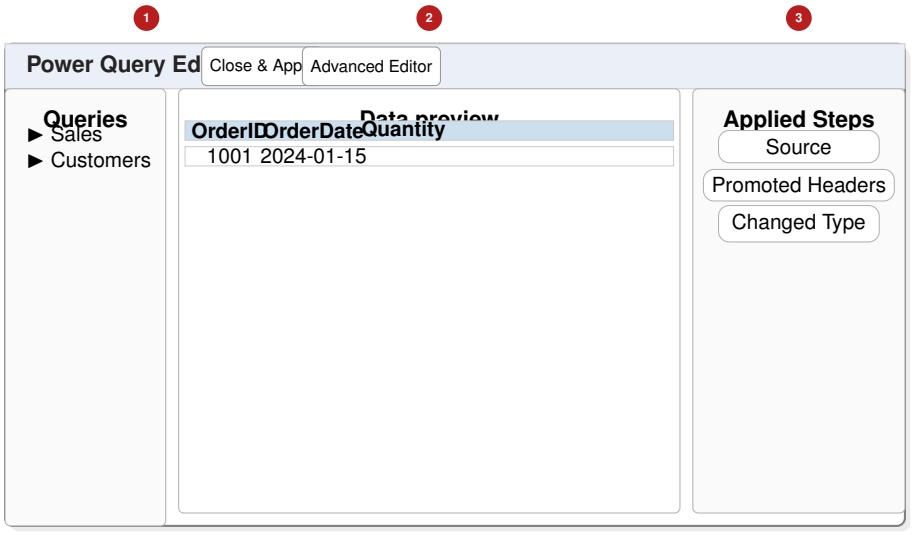


Figure 3.1: The Power Query editor (schematic). 1: the queries. 2: the data preview. 3: the applied steps, which form the recipe that Power Query replays on every refresh.

Panel	What it shows you
Queries (left)	One entry for each table or query. This is where you rename things.
Preview (middle)	The data as it looks after the steps you have applied so far.
Applied Steps (right)	The recorded recipe. Click any step to see the data at that point in the process.

Open the editor

Step 1. [Main Window] Click ***Home*** → ***Transform data***.

Step 2. [Power Query] Confirm that the Queries, Preview, and Applied Steps panels are all visible.

Check yourself

You can point to all three panels and explain what each one shows.

A word from your lecturer

Think of the Applied Steps panel as a diary of your cleaning. Each line records one decision. If something looks wrong later, you can click back through the diary to find the step where the problem started. Never delete steps you do not understand. Instead, click them and see what changed.

3.2 Rename queries

Rename the Sales query

Step 1. [Power Query] In the ***Queries*** pane, right-click ***Sales***, then choose ***Rename***.

Step 2. [Power Query] Type **Sales** and press Enter.

Step 3. [Power Query] Click ***Close & Apply***.

Why this matters: The name of a query becomes the name of the table in the model. Report authors will drag `Sales` into their visuals, so a clear name saves everyone time and confusion.

3.3 Parameterise the file path

Hard-coding a file path like `C:/Users/Me/Desktop/Sales.csv` into a model is a classic beginner mistake. The moment you move the file or publish the report, the path breaks. A parameter solves this by keeping the path in one place.

Create the FolderPath parameter and add LineAmount

Step 1. [Power Query] Click ***Home*** → ***Manage Parameters*** → ***New Parameter***.

Step 2. [Parameter Window] Name it `FolderPath`, set the Type to ***Text***, and set the Current Value to `C:/PowerBI_Course/data`. Click ***OK***.

Step 3. [Power Query] Select the ***Sales*** query, then click ***Home*** → ***Advanced Editor***.

Step 4. [Advanced Editor] Replace the existing code with the code below, then click ***Done***.

```
let
Source      = Csv.Document(File.Contents(FolderPath & "/Sales.csv"),
[Delimiter = ",", Encoding = 65001, QuoteStyle = QuoteStyle.Csv]),
Headers     = Table.PromoteHeaders(Source, [PromoteAllScalars = true]),
Typed       = Table.TransformColumnTypes(Headers, {
  {"OrderID", Int64.Type}, {"OrderDate", type date},
  {"CustomerKey", Int64.Type}, {"ProductKey", Int64.Type},
  {"Quantity", Int64.Type}, {"UnitPrice", Currency.Type}}),
LineAmount = Table.AddColumn(Typed, "LineAmount",
each [Quantity] * [UnitPrice], type number)
in
LineAmount
```

Let me explain the code briefly, since you will be asked about it. `Csv.Document` reads the file. `Table.PromoteHeaders` uses the first row as column names. `Table.TransformColumnTypes` sets each column's type. Finally, `Table.AddColumn` creates `LineAmount` by multiplying quantity by unit price for each row.

Why this matters: Changing one parameter is far safer than searching through every query to fix hard-coded paths. Forward slashes work in Windows paths and in cloud addresses alike, which is why we use them.

Check yourself

Sales now has a column called **LineAmount**. Select it and read the status bar at the bottom. The sum should be **2,065** before the 2023 file is appended.

If something goes wrong: An error such as "Token Comma Expected" usually means a stray character or a missing comma in the code. Compare your code line by line with the listing above.

3.4 Set data types

Data types decide what a column is allowed to do. A date stored as plain text cannot be sorted by time. A number stored as text cannot be added up. This is why cleaning types is not a cosmetic task. It decides whether your calculations can work at all.

Load Customers and Products with correct types

Step 1. [Power Query] Click ***Home*** → ***New Source*** → ***Text/CSV***, then open `Customers.csv`.

Step 2. [Power Query] Rename the query to **Customers**.

Step 3. [Power Query] Click the column header ***CustomerKey***. Click the small type icon to the left of the header, which shows `ABC` or `123`, and choose ***Whole Number***.

Step 4. [Power Query] Repeat the process for `Products.csv`. Rename the query **Products**. Set ***ProductKey*** to ***Whole Number***, and set ***UnitCost*** to ***Decimal Number***.

Step 5. [Power Query] Click ***Close & Apply***.

Why this matters: A relationship only works when the key columns on both sides have the same data type. If one side is text and the other is a number, the relationship looks correct on screen but matches nothing. This is the single most common cause of the beginner complaint "my visual shows zero."

Watch out

Check every key column on both sides before you build any relationship. The check takes about thirty seconds, and skipping it can cost you hours of confusion later.

Check yourself

Every key column shows the `123` icon rather than `ABC`.

3.5 Add a conditional column

A conditional column creates a new category based on a rule. It is one of the most useful tools you will use, because report readers often want to filter by categories that did not exist in the raw data.

Flag large orders

Step 1. [Power Query] Select the ***Sales*** query.

Step 2. [Power Query] Click ***Add Column*** → ***Conditional Column***.

Step 3. [Conditional Column Window] Name the new column `OrderSize`. If ***LineAmount*** is greater than or equal to `200`, the output is `Large`. Otherwise, the output is `Standard`. Click ***OK***.

Why this matters: The editor writes M code for you. In plain words, the rule says: if the line amount is at least 200, label the row Large; otherwise label it Standard. A report author can now filter the whole report by order size without any extra work.

Check yourself

Every row in the new column shows either `Large` or `Standard`.

3.6 Column from Examples

Sometimes a column contains a pattern that is hard to describe as a rule, such as a full name from which you want only the first name. Power Query can learn the rule from a few examples you type yourself.

Extract a first name

Step 1. [Power Query] Select the `CustomerName` column in the ***Customers*** query.

Step 2. [Power Query] Click ***Add Column*** → ***Column from Examples*** → ***From Selection***.

Step 3. [Power Query] In the first cell of the new column, type the first name that belongs to that row, for example `John` from "John Doe." Press Enter, and Power Query fills in the rest.

Step 4. [Power Query] Scroll through several rows, especially names with unusual formats. Rename the column `FirstName`.

Why this matters: Column from Examples writes the rule for you, so you do not need to know the text functions in advance. A rule learned from one example can still go wrong with unusual formats, so always check rows you did not type yourself.

3.7 Handling time zones

Many systems store timestamps in UTC, which stands for Coordinated Universal Time. A report for a local audience should show local time. The safest place to make that conversion is in Power Query, so that every visual in the report uses the same converted value.

Shift a UTC timestamp to Kigali time

Step 1. [Power Query] Select a query that contains a timestamp column called `OrderTimestampUTC`. Confirm that its type is ***Date/Time***, not ***Date***.

Step 2. [Power Query] Click ***Add Column*** → ***Custom Column***, and name the new column `OrderTimestampLocal`.

Step 3. [Custom Column Window] Enter this formula: `[OrderTimestampUTC] + #duration(0, 2, 0, 0)`. Click ***OK***.

Why this matters: The expression `#duration(days, hours, minutes, seconds)` is how M expresses a length of time. Adding two hours moves a UTC time to Central Africa Time, which is UTC+2 and is the zone used in Kigali. The column must be a date-time type, because a plain date has no time of day to shift.

Watch out

A fixed offset is wrong in countries that change their clocks for daylight saving time. For those countries, use a time-zone-aware conversion instead. Look up the `DateTimeZone` functions in the M reference and test your result on a date either side of a clock change. Kenya is UTC+3 and does not change its clocks, so the same method works there with three hours.

3.8 Append the 2023 data

Appending stacks the rows of one table on top of another. You use it when several files share the same columns, as our 2023 extension does.

Append Sales_2023_Extension to Sales

Step 1. [Power Query] Right-click ***Sales***, then choose ***Duplicate***. Rename the copy **Sales2023**.

Step 2. [Power Query] Select ***Sales2023***. Open ***Advanced Editor***, and change the file name in the `Source` line from `Sales.csv` to `Sales_2023_Extension.csv`. Click ***Done***.

Step 3. [Power Query] Select ***Sales***. Click ***Home*** → ***Append Queries*** → ***Two tables***.

Step 4. [Append Window] Set the first table to ***Sales*** and the second table to ***Sales2023***. Click ***OK***.

Step 5. [Power Query] Click ***Close & Apply***.

Why this matters: Append stacks rows, one table below another. Merge joins columns, placing matching rows side by side. Choosing the wrong one is one of the most frequent Power Query mistakes, so it is worth remembering: append adds rows, merge adds columns.

Check yourself

The combined Sales query contains every row from both files, every row has a value in LineAmount, and the total is 2,805 (2,065 from 2024 plus 740 from 2023).

Watch out

Appending tables whose columns do not match does not cause an error. Power BI fills the missing columns with blanks, and your totals change silently. Compare the column names and their types before every append.

3.9 Merge queries (joins)

Merging combines two tables side by side, matching rows on a shared key. If you know SQL, it is the same idea as a join.

Join kind	What it keeps
Left outer	Every row of the first table, with matching details from the second table where they exist.
Inner	Only the rows that match in both tables.
Left anti	Rows from the first table that have no match in the second table.
Full outer	Every row from both tables, matched where possible.

Find customers who have never ordered

Step 1. [Power Query] Select ***Customers***, then click ***Home*** → ***Merge Queries***.

Step 2. [Merge Window] Choose ***Sales*** as the second table. Select ***CustomerKey*** in both tables. Set the Join Kind to ***Left Anti***, then click ***OK***.

Step 3. [Power Query] The result lists the customers with no orders. Note how many there are. Then click the merge step in ***Applied Steps*** and delete it, so the query returns to its normal form.

Why this matters: A left anti join answers the question "who is missing?" Business teams ask this constantly, for example to find inactive customers, products that never sold, or stores that did not report.

Watch out

A merge adds a column that contains a nested table. You must expand that column deliberately. If you do not, you will have to explain the odd-looking rows to your report readers later.

3.10 Unpivot

Unpivoting turns columns into rows. It repairs the most common spreadsheet layout, where every month has its own column. A model needs one row per event, so a table with twelve month columns is awkward to filter and to chart.

Unpivot a monthly table

Step 1. [Power Query] Load a table with a `Product` column and one column per month, such as `Jan`, `Feb`, and `Mar`.

Step 2. [Power Query] Select the `Product` column. Click ***Transform*** → ***Unpivot Columns*** → ***Unpivot Other Columns***.

Step 3. [Power Query] Rename the two new columns to `Month` and `Amount`.

Why this matters: Once the months are rows, a single date filter or a single trend chart works for every month. You no longer need a separate column for each one.

3.11 Error handling

Errors happen. A typo lands in a number, or a date is missing. Your job is to decide what the report should do when that happens, rather than letting the error pass silently.

Handle errors deliberately

Step 1. [Power Query] Select the column that contains the errors.

Step 2. [Power Query] Click ***Home*** → ***Remove Errors*** to drop the error rows, or ***Replace Errors*** to set a default value.

Step 3. [Power Query] Click ***Close & Apply***.

Go deeper

In M you can handle errors one expression at a time with `try ... otherwise`. For example, `try Number.From([Qty]) otherwise 0` returns zero when the conversion fails. Be careful with this. Replacing an error with zero hides a data problem. A good practice is to count the errors first and report them to the data owner.

3.12 Query folding

Query folding is a concept that appears in exams and in real projects alike. It means Power Query translates your steps into the query language of the source system, such as SQL. The source then does the heavy work, which is usually much faster than pulling everything into Power BI first.

Watch out

Some steps stop folding. Buffering the table in memory is one example, and so is using a function that has no SQL equivalent. To check, right-click a step in Applied Steps. If ***View Native Query*** is greyed out, folding has stopped at that step. Move your filters earlier in the query so they run at the source.

Go deeper

Folding matters most when the source is a large database and when you use incremental refresh, which you will meet in Part 8. On a CSV file it makes no difference, because a CSV has no query engine to fold into.

Practice

1. Create a conditional column called `Segment Flag` on Customers. It should return `VIP` when Segment is “Enterprise” and `Standard` otherwise.
2. Merge Products into Sales on `ProductKey` using a left outer join. Expand only `ProductName` and `Category`.
3. Create a query that keeps only the 2023 rows, using a filter on `OrderDate`. Explain in one sentence why this step folds when the source is a database but not when it is a CSV file.

Solutions

1. Add Column → Conditional Column. The rule is: if Segment equals Enterprise, output VIP; otherwise output Standard.
2. Home → Merge Queries. Choose Sales and Products, select ProductKey in both, and set the kind to Left outer. Expand the new column and tick only ProductName and Category.
3. A date filter becomes a WHERE clause when the database does the work, so the database filters the rows before sending them. A CSV has no query engine, so Power BI filters the rows itself after loading them.

What you have just built. Clean, typed queries with a parameterised file path, a calculated column, a conditional category, an example-based column, a time-zone conversion, an appended 2023 extension, and a join example. The data is now ready to be related together.

Part 4

Model the Data

By the end of this part you will be able to

- Describe a star schema and state the grain of a fact table.
- Create relationships with the correct cardinality and filter direction.
- Build a continuous, marked date table.
- Handle inactive relationships, role-playing dates, and many-to-many relationships.

Modelling is the step that turns a pile of tables into a system that answers questions. It is also the part that examiners probe most often, because a good model makes the DAX simple, and a bad model makes almost every measure painful. Think of the model as the blueprint of a building. You would not start painting the walls before the foundations are poured.

4.1 The star schema

The most important pattern in Power BI modelling is the **star schema**. It places one **fact table** in the centre, holding the events or numbers you want to add up, and surrounds it with **dimension tables**, which describe those events.

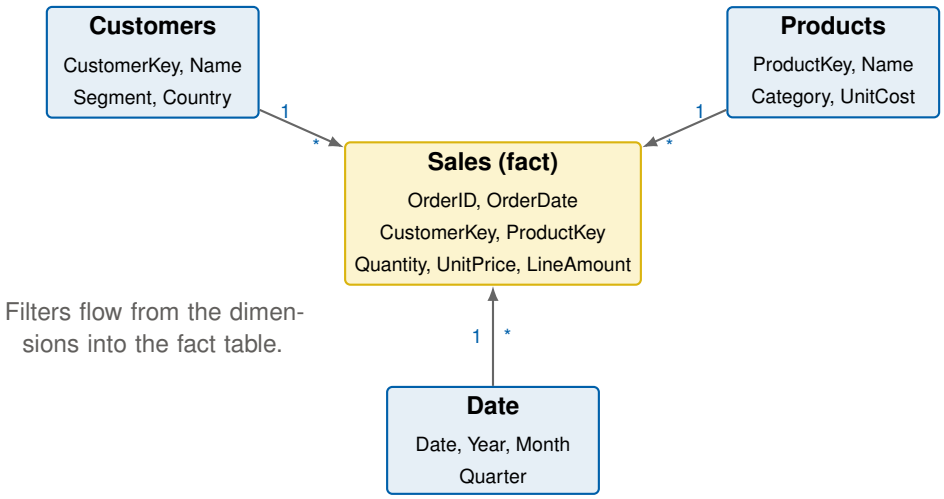


Figure 4.1: A star schema. One fact table at the centre, with dimension tables around it. The 1 marks the dimension side of each relationship, and the $*$ marks the fact side.

The grain is the most important word in this section. The grain of a fact table is what one row represents. In our sales table, one row is one product line on one order. Write the grain down for

every fact table you build. If you cannot state the grain in one sentence, you are not yet ready to model the table.

Filter direction is the second key idea. In a one-to-many relationship from a dimension to a fact table, filters flow from the dimension, which is the "one" side, into the fact table, which is the "many" side. So when a report reader selects Kenya in a slicer, the Country filter flows into Sales, and only Kenyan sales remain visible. The fact table never needs to store the country name on every row.

Go deeper

A **snowflake schema** normalises the dimensions further. For example, Products might link to a Subcategory table, which links to a Category table. This saves a little storage, but it makes the model harder for report authors to use. Power BI handles both designs well, and the star is the default recommendation for almost every report.

Practice

1. State the grain of a fact table that records one payment per invoice.
2. Your Customers table has a Region column that you need in a report on Sales. Should you copy the column into Sales, or should you relate the tables? Explain your answer.

Solutions

1. One row equals one payment. If an invoice can be paid in several instalments, the grain is the payment, not the invoice. Stating the grain correctly is what prevents double counting later.
2. Relate the tables. Copying the region into every sales row repeats the same text millions of times, wastes memory, and creates inconsistencies whenever a region is renamed.

4.2 Create the relationships

Link Sales to Customers and Products

Step 1. [Main Window] Click ***Model*** on the left edge of the window.
Step 2. [Model View] Drag ***Sales[CustomerKey]*** onto ***Customers[CustomerKey]***.
Step 3. [Edit Relationship Window] Check that the relationship reads ***Many to one (*:1)***, that the direction is ***Single***, and that ***Make this relationship active*** is ticked. Click ***Save***.
Step 4. [Model View] Drag ***Sales[ProductKey]*** onto ***Products[ProductKey]***.
Step 5. [Edit Relationship Window] Check the same three settings, then click ***Save***.

Why this matters: "Many to one" means many sales rows can point to one customer. "Single" means filters travel in one direction only, from the customer into the sales. Filtering in both directions creates ambiguous paths through the model, and it can inflate your totals.

Check yourself

Two solid lines connect the three tables. Each line shows a `1` at the dimension end and a `*` at the Sales end.

*If something goes wrong: A dashed line means the relationship is inactive. Open it and tick **Make this relationship active**. If you cannot drag a line at all, the key types probably differ. Return to Part 3 and make both sides Whole Number.*

Watch out

Do not set the direction to ***Both*** "just in case." It is the fastest route to doubled totals, and Power BI will not warn you. Use two-way filtering only when a specific design needs it, and write down the reason.

4.3 Build and mark the Date table

Time intelligence, such as year-to-date, year-over-year, and rolling totals, depends on a Date table that is continuous and marked. Continuous means every single day exists in the table, with no gaps. Marked means you have told Power BI which column is the official calendar.

Create the Date table

Step 1. [Main Window] Click ***Table*** on the left edge.

Step 2. [Table View] Click ***Table tools*** → ***New table***.

Step 3. [DAX Formula Bar] Paste the code below and press Enter.

```
Date =
ADDCOLUMNS(
CALENDAR(DATE(2023, 1, 1), DATE(2024, 12, 31)),
"Year",          YEAR([Date]),
"Month Number", MONTH([Date]),
"Month",         FORMAT([Date], "mmm"),
"Year-Month",    FORMAT([Date], "yyyy-mm"),
"Quarter",       "Q" & QUARTER([Date])
)
```

Sort the months and mark the table

Step 1. [Table View] Select the **Date** table, then click the ***Month*** column header.

Step 2. [Column Tools] Click ***Sort by column*** → ***Month Number***.

Step 3. [Table View] Select the **Date** table. Click ***Table tools*** → ***Mark as date table***, choose the ***Date*** column, and click ***OK***.

Step 4. [Model View] Drag ***Sales[OrderDate]*** onto ***Date[Date]***. Check that it is Many to one, Single, and active.

Why this matters: Sorting by Month Number stops January from appearing after December, which alphabetical sorting would do. Marking the table tells Power BI that this is the official calendar, so time-intelligence functions behave correctly.

Check yourself

A chart with `Date[Month]` on the axis shows January to December in the right order, not Apr, Aug, Dec, Feb.

Watch out

If the calendar does not cover every order date, or if it has gaps, time-intelligence measures return blanks or misleading growth figures without any error. Check that the calendar spans every date in both Sales and Sales2023.

4.4 Role-playing dates

A sales table often holds several dates: the order date, the ship date, and the delivery date. Each one relates to the same Date table. However, only one relationship can be active at a time. The others stay inactive and are used only when a measure asks for them explicitly. This pattern is called a role-playing date.

Add an inactive ShipDate relationship

Step 1. [Model View] Drag ***Sales[ShipDate]*** onto ***Date[Date]***. In the Edit Relationship window, untick ***Make this relationship active***, then click ***Save***.

Step 2. [Model View] The solid line stays on OrderDate. The new dashed line is the inactive ShipDate relationship.

Why this matters: Power BI allows only one active path between two tables, because two active paths would make every filter ambiguous. The inactive relationship waits in the background until a measure asks for it with `USERELATIONSHIP`*, which you will write in Part 5.*

4.5 Many-to-many relationships

Sometimes neither side of a relationship has unique values. A customer might belong to several segments, and a segment might contain many customers. A direct relationship between such tables produces wrong totals. The standard fix is a bridge table, which holds one row for every unique pairing.

Watch out

Power BI will let you create a many-to-many relationship between two non-unique columns, and then it can double-count without complaint. Build a bridge table with unique combinations, and always check your totals against a figure you already know.

4.6 Hide, organise, and categorise

Good models are generous to the people who use them. Hiding technical columns, such as keys, keeps the field list short and friendly for report authors.

Tidy the model for report authors

Step 1. [Report View] In the Data pane, right-click ***Sales[OrderID]***, then choose ***Hide in report view***. Do the same for ***CustomerKey*** and ***ProductKey***.

Step 2. [Table View] Select **Customers**, then click the ***Country*** column. Under ***Column tools***, set ***Data category*** to ***Country***.

Step 3. [Report View] In the Data pane, right-click ***Date[Year]***, then choose ***Create hierarchy***. Drag ***Month*** and ***Date*** into the new hierarchy.

Why this matters: Hiding is about presentation, not security. A hidden column still works perfectly in relationships and in DAX. Setting a data category tells Power BI that Country is a place, so it can draw a map. A hierarchy lets a report reader drill from year to month to day with one click.

Watch out

Hiding a column does not protect it. Anyone who can open the model can still see it. Use row-level security or object-level security, which you will meet in Part 7, to protect data.

Go deeper

Slowly changing dimensions. A Type 1 dimension simply overwrites old values. A Type 2 dimension keeps the history. Suppose a customer moves from Uganda to Kenya. A Type 2 table gives them a new row with a new surrogate key, a start date, an end date, and the same business key that identifies the real-world customer. To count customers correctly, you must count distinct business keys, not surrogate keys. Part 12 includes a question on exactly this trap.

Practice

1. Create the relationship between Sales and Products on `ProductKey`. Confirm that it is many to one and single direction.
2. Add a hierarchy called `Calendar` to the Date table, with the levels Year, Quarter, Month, and Date, in that order.
3. Hide the `UnitCost` column in Products from the report view. Explain whether a measure can still use it, and why.

Solutions

1. Model view, drag `Sales[ProductKey]` onto `Products[ProductKey]`, then confirm Many to one and Single.
2. Right-click Year, choose Create hierarchy, then drag Quarter, Month, and Date into it in that order.
3. Hiding removes the column from the field list only. A measure such as `SUMX(Sales, Sales[Quantity] * RELATED(Products[UnitCost]))` still works, because hidden columns remain part of the model.

What you have just built. A star schema with three relationships, a continuous and marked date table, an inactive ship-date relationship, and a tidy field list that report authors will thank you for.

Part 5

DAX: Calculating Answers

By the end of this part you will be able to

- Understand Row Context and Filter Context.
- Write error-free measures based on your Star Schema.
- Use `CALCULATE` to change filters, and `VAR` to make code readable.

DAX (Data Analysis Expressions) is where the real analytical power of Power BI lives. Think of DAX as Excel formulas on steroids. In this chapter, we will write every measure you need for your model, explaining exactly what each piece of code does so you can reproduce it from memory.

5.1 The Two Contexts: How DAX Thinks

Before writing code, you must understand the two ways DAX evaluates data.

- **Row Context:** This means "look at one row at a time." It happens when you create a *Calculated Column* or use an iterator function like `SUMX`. DAX goes row by row, does the math, and then moves to the next row.
- **Filter Context:** This is the "environment" your calculation is currently in. If you put a slicer for "Kenya" on the page, the Filter Context is restricted to Kenya. `CALCULATE` is the only function that can change the Filter Context.

⚠ Watch out

Do not create Calculated Columns for totals (e.g., a column that sums sales). Calculated Columns take up memory and do not respond to slicers. Always use **Measures** for totals.

5.2 Step 1: The Base Measures

Look at your Model View. Your `sales` table contains `Quantity` and `UnitPrice`. Your `Products` table contains `UnitCost`.

Because `UnitCost` is in a different table, we use the `RELATED` function to pull it across the relationship line.

Create the core financial measures

Step 1. [Main Window] Select the **sales** table in the Data pane, then click ***Home*** → ***New measure***.

Step 2. [DAX Formula Bar] Rwite the code below after reading the green comments to understand exactly what is happening.

```
-- 1. TOTAL SALES
-- We use SUMX to multiply Quantity by UnitPrice row-by-row, then add it all up.
Total Sales =
SUMX(sales, sales[Quantity] * sales[UnitPrice])

-- 2. TOTAL COST
-- We use RELATED to fetch the UnitCost from the Products table for each row.
Total Cost =
SUMX(sales, sales[Quantity] * RELATED(Products[UnitCost]))

-- 3. GROSS PROFIT
-- We subtract one measure from another. Notice the square brackets!
Gross Profit =
[Total Sales] - [Total Cost]

-- 4. GROSS MARGIN %
-- DIVIDE handles division safely. If Total Sales is 0, it returns BLANK instead of an
   error.
Gross Margin % =
DIVIDE([Gross Profit], [Total Sales])

-- 5. TOTAL ORDERS
-- DISTINCTCOUNT counts unique OrderIDs. It prevents double-counting if an order has
  multiple items.
Total Orders =
DISTINCTCOUNT(sales[OrderID])
```

Check yourself

Create a Card visual and drag `[Total Sales]` and `[Gross Profit]` onto it. Your numbers should match the sum of your CSV files. If you see an error, check that your relationship lines in Model View are connected properly.

5.3 Step 2: Filtering with CALCULATE

`CALCULATE` is the most powerful function in DAX. It changes the Filter Context. It does four things:

1. Takes the existing filters (from slicers/charts).
2. Replaces them with the filters you write in the formula.
3. Evaluates the measure.
4. Returns the result.

Write filter measures

Step 1. [Main Window] Create a new measure on the **sales** table.

```
  -- 1. BASIC FILTER
  -- We force DAX to only look at rows where the Country is Kenya.
  Sales Kenya =
  CALCULATE([Total Sales], Customers[Country] = "Kenya")

  -- 2. MULTIPLE FILTERS
  -- You can add commas. This means Country = Kenya AND Category = Hardware.
  Kenya Hardware Sales =
  CALCULATE(
  [Total Sales],
  Customers[Country] = "Kenya",
  Products[Category] = "Hardware"
  )
```

```
\begin{lab}{Compare CALCULATE with and without KEEPFILTERS}
  \step[Main Window] Create two measures on the \textbf{sales} table.
\end{lab}

\begin{lstlisting}[language=DAXlang]
  -- 1. OVERRIDE (Without KEEPFILTERS)
  -- Ignores the visual. ALWAYS shows Kenya sales.
  Sales Kenya (Override) =
  CALCULATE(
  [Total Sales],
  Customers[Country] = "Kenya"
  )

  -- 2. INTERSECT (With KEEPFILTERS)
  -- Respects the visual. Shows BLANK if the row is not Kenya.
  Sales Kenya (Intersect) =
  CALCULATE(
  [Total Sales],
  KEEPFILTERS(Customers[Country] = "Kenya")
  )
```

Why this matters: The Main Difference

Without `KEEPFILTERS`, the formula **replaces** the visual's filter. With `KEEPFILTERS`, it **combines** with it. If the visual is showing Uganda, the Intersect measure returns blank because a row cannot be both Uganda and Kenya.

Check yourself

Create a Table visual with three columns:
`Customers[Country]`, `[Sales Kenya (Override)]`, and `[Sales Kenya (Intersect)]`.
Look at the **Uganda** row: Override still shows Kenya's sales number, but Intersect shows **BLANK**.

Watch out

`CALCULATE` only overrides the exact column you name (Country). Any other filters on the page—like a Date slicer—will still apply normally.

Watch out

This is the exam question. Without `KEEPFILTERS`, your filter **overrides** the visual. With `KEEPFILTERS`, your filter **intersects** with the visual. If you need a measure to respect what the user clicked on a slicer, use `KEEPFILTERS`.

5.4 Step 3: Variables (VAR) and RETURN

When formulas get long, they are hard to read. Use `VAR` to store a result once, and `RETURN` to output the final answer. It also makes your DAX run faster because Power BI only calculates the variable once.

Rewrite a measure with variables

Step 1. [Main Window] Create a new measure on the **sales** table.

```
-- We store the margin and the target, then check them.
Margin Status =
VAR CurrentMargin = [Gross Margin %]
VAR TargetMargin = 0.40
RETURN
IF(CurrentMargin >= TargetMargin, "On Target", "Below Target")
```

5.5 Step 4: Advanced Analysis (Top Performers)

To find the "Top Customer" or "Top Country", we have to temporarily remove the filters from the visual using `ALL()`, find the maximum value, and then return the name of the winner.

Find the country with the most customers

Step 1. [Main Window] Create a new measure on the **sales** table.

```
-- Find the country (or countries) with the highest number of unique clients.
Top Country(s) by Customers =
-- 1. Find the highest number of unique clients across all countries.
VAR MaxClients =
MAXX(
ALL(Customers[Country]),
CALCULATE(DISTINCTCOUNT(sales[CustomerKey]))
)

-- 2. Return the country name(s) whose client count equals that maximum.
RETURN
CONCATENATEX(
FILTER(
ALL(Customers[Country]),
CALCULATE(DISTINCTCOUNT(sales[CustomerKey])) = MaxClients
),
Customers[Country],
", " -- Separate country names with commas if there is a tie.
)
```

Go deeper

Why use `ALL`? If you put this measure in a table visual with Customer Names on the rows, DAX would normally calculate a "Top Country" for every single customer row. By wrapping the country column in `ALL()`, we tell DAX: *"Ignore the Customer rows on the visual, look at the whole dataset, and find the global winner."*

5.6 Step 5: Time Intelligence

Time intelligence requires your `Date` table to be marked as a Date Table (which you did in Model View).

Calculate Year-to-Date and Previous Month

Step 1. [Main Window] Create these measures on the **sales** table.

```
-- 1. YEAR TO DATE (YTD)
-- Adds up Total Sales from January 1st up to the current date in the filter.
Sales YTD =
TOTALYTD([Total Sales], 'Date'[Date])

-- 2. PREVIOUS MONTH (PM)
-- Shifts the date filter back by exactly one month.
Sales PM =
CALCULATE([Total Sales], DATEADD('Date'[Date], -1, MONTH))

-- 3. MONTH-OVER-MONTH GROWTH %
-- Calculates the percentage difference between this month and last month.
MoM Growth % =
DIVIDE([Total Sales] - [Sales PM], [Sales PM])
```

A word from your lecturer

If `[Sales PM]` is blank, don't panic! It means there is no data for the previous month in your dataset. For example, January will always have a blank Previous Month because there is no December before it.

5.7 Practice: Test Your Knowledge

Practice

1. Create a measure called `Uganda Share` that calculates Uganda's Total Sales as a percentage of all Total Sales (ignoring any slicers).
2. Create a measure called `Top Buyer by Volume` that finds the `CustomerName` who bought the highest total `Quantity`. Use the `TOPN` pattern from Step 4.

Solutions

1.
```
Uganda Share =
DIVIDE(
```

```
    CALCULATE([Total Sales], Customers[Country] = "Uganda"),
    CALCULATE([Total Sales], ALL(Customers))
    )
```

2.

```
    Top Buyer by Volume =
    VAR CustomerStats =
    ADDCOLUMNS(
    ALL(sales[CustomerName]),
    "@TotalQty", SUM(sales[Quantity])
    )
    VAR Winner =
    TOPN(1, CustomerStats, [@TotalQty], DESC)
    RETURN
    CONCATENATEX(Winner, sales[CustomerName], ", ")
```

Part 6

Visualise and Analyse

By the end of this part you will be able to

- Choose a chart type that matches the question being asked.
- Build an overview page with a working slicer.
- Use edit interactions, drillthrough, tooltips, and bookmarks.
- Apply design rules that keep a report readable for everyone, including colour-blind viewers.

A report is a conversation between the data and the reader. Each visual answers one question, and interactions let the reader ask a follow-up without leaving the page. When you design a page, start by writing down the question it must answer. Then choose the visual. Decorations come last, if at all.

6.1 Choose the right chart

The question	The visual to use
One number that matters	Card, or KPI
How categories compare	Bar or column chart
How something changes over time	Line chart
How two numbers relate to each other	Scatter chart
The exact value of many items	Table or matrix
How the parts make up a whole	Donut or pie, only with five parts or fewer

Go deeper

Cleveland and McGill studied how accurately people read different visual encodings. Position along a common scale is read most accurately, followed by length, angle, area, and colour. Bars use position and length. Pies use angle and area. That explains why bar charts are almost always easier to read than pie charts, and why a pie with many slices is so hard to use.

6.2 Build the Overview page

Build the Overview page

Step 1. [Report View] Double-click the page tab at the bottom of the window, and rename the page **Overview**.

Step 2. [Visualizations] Click the ***Card*** icon. Drag ***Total Sales***, ***Gross Margin %***, and ***Orders*** into three separate cards.

Step 3. [Visualizations] Click the ***Line and clustered column chart*** icon. Drag ***Date[Month]*** to the X-axis, ***Total Sales*** to the column values, and ***Sales PM*** to the line values.

Step 4. [Visualizations] Click the ***Clustered bar chart*** icon. Drag ***Products[Category]*** to the Y-axis and ***Total Sales*** to the X-axis.

Check yourself

The cards should show Total Sales, Gross Margin, and Orders. The bar chart's largest category should be Equipment. Compare these numbers with the checkpoint table at the start of the course.

6.3 Add a slicer and edit interactions

A slicer is a visible filter that the reader clicks. By default it filters every visual on the page. Sometimes that is exactly what you want, and sometimes it is not. Edit interactions lets you decide which visuals respond.

Add a slicer that filters only one visual

Step 1. [Visualizations] Click the ***Slicer*** icon. Drag ***Products[Category]*** into it.

Step 2. [Report View] Select the slicer, then click ***Format*** → ***Edit interactions***.

Step 3. [Report View] Small icons appear above your other visuals. On the bar chart, click the ***None*** icon so the bar chart ignores the slicer. Leave the ***Filter*** icon on the line chart.

Why this matters: Edit interactions gives you control over each visual. Filter means the visual responds by narrowing its data. None means it ignores the slicer. Highlight is a middle option, which dims the parts that do not match rather than removing them.

Check yourself

Selecting a category now changes the line chart, and it leaves the bar chart unchanged.

6.4 Drillthrough

Drillthrough lets a reader right-click something interesting and jump to a detail page that is already filtered to that item.

Build a customer detail page

Step 1. [Report View] Add a new page and rename it **Customer Detail**.

Step 2. [Visualizations] Drag ***Customers[CustomerName]*** into the ***Drill through*** well.

Step 3. [Visualizations] Add a ***Table*** visual with ***OrderID***, ***OrderDate***, and ***Total Sales***.

Step 4. [Overview Page] Right-click a customer name in a table, then choose ***Drill through*** → ***Customer Detail***.

Why this matters: Drillthrough carries the current filter across to the new page. The detail page then shows only the customer that was clicked, and a back arrow appears so the reader can return.

Check yourself

Drilling through a customer shows only that customer's orders on the detail page.

6.5 Tooltips and bookmarks

A **tooltip** shows extra detail when a reader hovers over a visual. A **bookmark** saves the current state of a page, including its filters and slicer choices, so the reader can return to that view with one click.

Create two bookmarks and a navigation button

Step 1. [Report View] Select Equipment in the category slicer. Click ***View*** → ***Bookmarks*** → ***Add***, and rename the bookmark **Equipment view**.

Step 2. [Report View] Clear the slicer, then add a second bookmark named **All products**.

Step 3. [Insert] Click ***Insert*** → ***Buttons*** → ***Blank***. Set the button's action to ***Bookmark***, then choose ***Equipment view***.

Why this matters: A bookmark stores the page state, meaning its filters, slicer choices, visibility, and drill level. It does not store the data itself. After a refresh, the bookmark reapplies its filters to the new data.

Watch out

A bookmark that stops working after you rename a visual is usually pointing at a visual that no longer exists. Open the Selection pane to check the names, and rebuild the bookmark if necessary.

6.6 Design rules for readable reports

These five rules will make your reports better straight away.

1. Put the most important number in the top left corner, and give every visual a title that states the question it answers.
2. Use one accent colour to draw attention, and grey out everything else.
3. Never rely on colour alone. Add labels, icons, or patterns, because roughly one man in twelve has some form of colour-vision deficiency.
4. Start bar charts at zero. A truncated axis exaggerates small differences and misleads the reader.

5. Remove decoration that carries no meaning, such as 3D effects, heavy gridlines, and legends that repeat the axis labels.

Practice

1. Build a line chart of Total Sales by Month, and add Sales LY as a second line. Check that the two lines line up month by month.
2. Add a second slicer for Customers[Country]. Set its interactions so it filters the cards but not the bar chart.
3. Create a drillthrough page for Products. Drill through from the bar chart, and show each product's orders.
4. Create a tooltip page that displays Gross Margin % and Orders when a reader hovers over a bar.

Solutions

1. Add Date[Month] to the axis, and add Total Sales and Sales LY as values. Both lines use the same month positions, so they line up.
2. Select the country slicer, then choose Format → Edit interactions. Set the bar chart to None and leave the cards on Filter.
3. Add a page, drag Products[ProductName] into the drill-through well, add a table of orders, then right-click a bar and choose Drill through.
4. Create a new page, set its type to Tooltip under Format page, and add the two measures as cards. On the bar chart, open the tooltip settings and select that page.

What you have just built. An overview page with cards, a trend, a category comparison, a working slicer, drillthrough, tooltips, and bookmark-driven navigation.

Part 7

Publish, Secure, and Refresh

By the end of this part you will be able to

- Publish a report to a workspace and share it as an app.
- Assign the correct workspace role to each person.
- Configure a scheduled refresh, and know when a gateway is needed.
- Build static and dynamic row-level security, and test it correctly.

Building a report is only half the job. The other half is making sure the right people see the right data, and that the numbers stay current. This part covers both, and it contains the security facts that the certification exam returns to again and again.

7.1 Workspaces, apps, and roles

Authors work in a **workspace**. Consumers, the people who simply read the report, see a published **app**. The two are kept apart on purpose. Authors can revise their work without disturbing anyone who is using the report.

Role	View	Edit	Publish app	Manage access	Subject to RLS
Viewer	Yes	No	No	No	**Yes**
Contributor	Yes	Yes	No	No	No
Member	Yes	Yes	Yes	Yes	No
Admin	Yes	Yes	Yes	Yes	No

Watch out

Row-level security applies only to Viewers. Contributors, Members, and Admins see every row, whatever roles you have defined. This is one of the most frequently tested security facts in the certification exam, so please memorise it.

7.2 Publish the report

Publish and assign a Viewer

Step 1. [Main Window] Save the file as `DataHills_Sales.pbix`, then click ***Home*** → ***Publish***.

Step 2. [Publish Window] Choose a workspace. ***My workspace*** is fine for practice. Click ***Select***.

Step 3. [Power BI Service] Open the workspace, click ***Manage access***, and add a second test account with the ***Viewer*** role.

Check yourself

The Viewer account can open the report, but it cannot edit it.

7.3 Refresh and gateways

A scheduled refresh reloads the data at a time you choose. The Service uses the stored credentials of each data source. It does not use the credentials of the person who published the report, which is a point examiners like to test.

Set up a cloud source with a scheduled refresh

Step 1. [External] Upload the four CSV files to OneDrive or SharePoint.

Step 2. [Power BI Desktop] Click ***Transform data***. Change the `FolderPath` parameter to the cloud folder address, click ***Close & Apply***, and publish again.

Step 3. [Power BI Service] Open the semantic model, then choose ***Settings***. Under ***Data source credentials***, sign in. Under ***Scheduled refresh***, turn it on and add failure notifications.

*Why this matters: A cloud source uses cloud credentials, which the Service can manage on its own. An on-premises source, such as a database inside a company network, needs a **gateway**. The gateway is the bridge between the cloud service and the internal network. When a report works in Desktop but fails in the Service, the cause is almost always a file path that points to your own computer.*

Watch out

A local file path works in Desktop and fails in the Service. Use cloud locations, or a gateway for internal sources, for any report that refreshes on a schedule.

7.4 Static row-level security

Row-level security, or RLS, controls which rows each role can see. The filter should sit on a dimension table, because filters flow from dimensions into facts. A filter placed on the fact table would not reach the dimension, so the names of other customers would still be visible.

Create the KenyaOnly role

Step 1. [Main Window] Click ***Modeling*** → ***Manage roles*** → ***Create***.

Step 2. [Manage Roles Window] Name the role `KenyaOnly`. Select the **Customers** table, and enter the filter `[Country] = "Kenya"`. Click ***Save***.

Step 3. [Main Window] Click ***Modeling*** → ***View as***, select ***KenyaOnly***, and click ***OK***.

Why this matters: The filter sits on Customers because the relationship flows from Customers into Sales. Once Customers is filtered, the restriction flows through to every sales row that belongs to a Kenyan customer.

Check yourself

Total Sales shows the Kenya total only. No Uganda or Tanzania customer appears in any list. A yellow bar at the top says you are viewing as KenyaOnly. Click ***Stop viewing*** to leave that mode.

If something goes wrong: If you still see Ugandan customers, check the yellow bar. If it does not mention the role, the role was not saved properly.

7.5 Dynamic row-level security

Static roles become tedious once you have many countries and many managers, because you would need a new role for each one. Dynamic RLS solves this with one role and a table that maps each user to the countries they may see.

There is one design trap here, and it is worth understanding properly. The mapping table holds one row per user and country, so its Country column repeats values. The Customers table also repeats countries, because many customers live in each country. Relating two tables that both repeat values produces a many-to-many relationship, and that gives unreliable results. The fix is to route the relationship through a small table that holds each country exactly once.

Build a dynamic role through a Countries table

Step 1. [Table View] Create a table called **UserCountry** with two columns, `UserEmail` and `Country`. Enter one row for each user and each country they may see.

Step 2. [Table View] Create a calculated table called **Countries** with the formula `Countries = DISTINCT(Customers[Country])`. Each country now appears exactly once.

Step 3. [Model View] Create a relationship from **Countries[Country]**, on the one side, to **Customers[Country]**, on the many side. Set the direction to ***Single***.

Step 4. [Manage Roles] Create a role called `DynamicCountry` on the **Countries** table, with this filter: `Countries[Country] IN CALCULATETABLE(VALUES(UserCountry[Country]), UserCountry[UserEmail] = USERPRINCIPALNAME())`.

Step 5. [Modeling] Click ***View as***. Test with one manager's address and check the total, then test with a second manager's address and check that total.

Why this matters: `USERPRINCIPALNAME()` returns the signed-in user's identity, which is reliable in the Service. The filter reads that user's allowed countries from the mapping table, and then applies them to the unique Countries table. From there the restriction flows into Customers and then into Sales. Avoid `USERNAME()` in new designs, because it behaves differently in Desktop and in the Service.

Watch out

Do not relate **UserCountry[Country]** directly to **Customers[Country]**. Both sides repeat values, so the relationship becomes many-to-many, and the security results can be wrong. Route the relationship through the unique Countries table, as shown above.

Watch out

Testing RLS with your own account proves nothing. You are probably an Admin or a Member, and those roles bypass RLS completely. Test with a Viewer account in the Service. This is the most common RLS mistake that beginners make, and the examiner knows it.

Go deeper

Object-level security (OLS), configured in Tabular Editor, hides whole tables or columns from a role. Use it when a role must not even see `UnitCost`. Use RLS when a role may see a table but only some of its rows. **Sensitivity labels** from Microsoft Purview classify content and can restrict exporting. **Certified** content has been reviewed by a designated certifier, while **promoted** content has been marked by its owner as ready for others to use. To test RLS outside the Service, DAX Studio is a free tool that connects to models and runs queries. Check its current documentation for role testing, and always confirm the final result with a Viewer account in the Service.

Practice

1. Create a static role called `UgandaOnly`, test it with View as, and compare the total with a figure you calculate yourself from the source file.
2. Add a Viewer account to your workspace. Open the report as that account and verify that it cannot edit the report.
3. Explain why RLS on Sales alone would not stop a Ugandan manager from seeing the names of Kenyan customers.

Solutions

1. Manage roles, then Create. Name the role `UgandaOnly`, select Customers, and filter `[Country] = "Uganda"`. Use View as to check the total against your own calculation.
2. In the Service, open Manage access and add the account with the Viewer role. Sign in as that account. The Edit button should be missing.
3. A filter on Sales limits the sales rows, but it does not filter Customers. Customer names from Kenya would still appear in any list built from the Customers table. Filtering the dimension restricts every table that relates to it.

What you have just built. A published report, a scheduled refresh, a static and a dynamic row-level security role, and a correct test performed with a Viewer account.

Part 8

Performance and Tooling

By the end of this part you will be able to

- Name the four cost centres of a Power BI model and find the slow one.
- Explain how cardinality drives model size.
- Use Performance Analyzer, and understand what DAX Studio and VertiPaq Analyzer report.
- Choose between Import, DirectQuery, and Direct Lake.

A report that takes thirty seconds to load will not be used, however beautiful it looks. Performance is a skill of diagnosis. The most common beginner mistake is to change something at random and hope it helps. Instead, find the bottleneck first, then fix that one thing.

8.1 Four cost centres

Cost centre	A typical cause
Refresh time	A slow source, poor query folding, or very large tables
Model size	High-cardinality columns, such as timestamps or free text
Formula engine	Heavy iterators, repeated CALCULATE calls, or complicated filters
Visual count	Too many visuals on one page, each sending its own query

Name the cost centre before you change anything. Changing the wrong thing wastes your time and sometimes makes things worse.

8.2 Cardinality

Memory cost depends on the number of *distinct values* in a column, not on the number of rows. A Country column with a handful of values compresses to almost nothing. A timestamp with millions of distinct values is expensive. This is why removing one high-cardinality column can save more memory than removing many low-cardinality ones.

Record a Performance Analyzer trace

Step 1. [Report View] Click *View* → *Performance analyzer*, then click *Start recording* and *Refresh visuals*.

Step 2. [Performance Analyzer Pane] Sort the results by *Duration*, then expand the slowest entry.

Step 3. [Performance Analyzer Pane] Compare the *DAX query* time with the *Visual display* time.

Why this matters: A long DAX query time points to your measures or your model. A long visual display time points to formatting, or to too many visual elements on the page. The split tells you where to look next.

Go deeper

DAX Studio connects to an open model, runs queries, and reports server timings. It separates the time spent in the storage engine from the time spent in the formula engine. **VertiPaq Analyzer** reports, for each column, the number of distinct values and the bytes it uses. **Aggregation tables** pre-summarise high-volume data for coarse queries. **Incremental refresh** reloads only recent partitions, which matters greatly for large fact tables.

8.3 Storage modes

Mode	When to use it
Import	The default. Queries are fast, the data is copied into the model, and it refreshes on a schedule.
DirectQuery	Use it when the data is too large to import, or when it must be live. Every visual sends a query to the source, so it is usually slower.
Direct Lake (Fabric)	Use it with Delta tables in OneLake. Speed is close to Import, and the data is not copied.

Watch out

DirectQuery is not faster simply because nothing is copied. It moves the work to the source system, and each visual then waits for that system to respond. Import is the right default unless you have a clear reason to choose otherwise.

Practice

1. Use Table view to find the column in Sales with the most distinct values. Propose one change that would reduce its cardinality.
2. Run Performance Analyzer on the Overview page. Which visual is slowest? Is its time spent in the DAX query or in the display?
3. A colleague suggests DirectQuery to make the report faster. Write two sentences explaining why you would question that suggestion.

Solutions

1. A timestamp column usually has the most distinct values. Round it to the date, or remove the time portion, if the time of day is not needed for analysis.
2. The answer depends on your model. If DAX query time dominates, look at your measures. If visual display time dominates, look at how many visuals you have and how heavily they are formatted.
3. DirectQuery sends a query to the source for every visual, so it usually slows the report down. It is justified only when the data is too large to import or must be live.

What you have just built. A model that you can measure, explain, and tune, using the right storage mode for your data.

Part 9

Building an Organizational Dashboard

By the end of this part you will be able to

- Understand why Power BI is superior to Excel for organizational dashboards.
- Design a report using the "Z-Pattern" for executive readability.
- Build a multi-page report with an Executive Summary and an Operational Detail view.
- Add professional polish using tooltips, bookmarks, and visual interactions.

You might be wondering: *"I know Excel can make a dashboard. Why should I use Power BI?"*

Excel is fantastic for a quick chart on a single sheet. But an organizational dashboard requires handling large datasets, secure sharing, automatic refreshes, and seamless interactivity. Power BI was built exactly for this. It is an engine for insights, not just a canvas for numbers.

In this chapter, we will take everything you have built—the cleaned data, the Star Schema, the DAX measures—and assemble it into a polished dashboard that a CEO or a Regional Manager would be thrilled to use.

9.1 The Golden Rules of Dashboard Design

Before you drop a single visual onto the canvas, you must understand how the human eye reads a screen.

A word from your lecturer

The Z-Pattern: People read a dashboard in a "Z" shape. They start at the top-left, move across to the top-right, drop down diagonally to the bottom-left, and finish at the bottom-right. Place your most important numbers (the KPIs) in the top-left.

Three rules for professional dashboards:

1. **Less is more.** Do not put 20 visuals on one page. If a visual does not answer a specific business question, delete it.
2. **One accent colour.** Use grey for everything, and use one strong colour (like Blue or Orange) to highlight what matters most.
3. **No chart junk.** Remove 3D effects, heavy gridlines, and unnecessary legends. Let the data speak.

9.2 Building the Executive Summary Page

The Executive page is for managers who need the big picture in five seconds. They don't want to see individual order IDs; they want to see trends and totals.

Create the Executive Overview

Step 1. [Report View] Rename Page 1 to **Executive Overview**.

Step 2. [Visualizations] Add three ***Card*** visuals at the very top. Drag ***[Total Sales]***, ***[Gross Profit]***, and ***[Gross Margin %]*** into them.

Step 3. [Visualizations] Add a ***Line Chart*** below the cards. Drag ***Date[Month]*** to the X-axis and ***[Total Sales]*** to the Y-axis.

Step 4. [Visualizations] Add a ***Donut Chart*** next to the line chart. Drag ***Products[Category]*** to the Legend and ***[Total Sales]*** to the Values.

Step 5. [Visualizations] Add a ***Slicer*** for ***Customers[Country]*** and place it at the top right.

Why this matters: Notice how we used Cards for absolute numbers, a Line chart for time trends, and a Donut for composition. The Country slicer sits at the top right, allowing the manager to instantly filter the entire page for Kenya, Uganda, or Tanzania.

Check yourself

The cards should show your Total Sales and Gross Margin. Clicking a different country in the slicer should instantly update the Line Chart and Donut Chart.

9.3 Building the Operational Detail Page

The Executive page gives the *what*. The Operational page gives the *why*. If the Executive sees that sales in Kenya dropped, they should be able to click a button and jump to the detail page to see which specific customers stopped buying.

Create the Operational Detail Page

Step 1. [Report View] Add a new page at the bottom and rename it **Operational Detail**.

Step 2. [Visualizations] Add a ***Matrix*** visual. Put ***Customers[CustomerName]*** on the Rows, ***Products[Category]*** on the Columns, and ***[Total Sales]*** in the Values.

Step 3. [Visualizations] Add a ***Table*** visual below the matrix. Include ***OrderID***, ***OrderDate***, ***[Total Quantity]***, and ***[Total Sales]***.

Check yourself

Click on a specific country in the slicer on the Executive page. Notice how the Matrix and Table on the Operational page update automatically. This is the power of Power BI—every visual on every page is connected to the same underlying data model.

9.4 Adding Professional Polish

To make your dashboard look like it was built by a senior data analyst, you need to add a few final touches.

9.4.1 Navigation Buttons

Instead of making the user click the tabs at the bottom, give them a button on the canvas.

Add Page Navigation

Step 1. [Insert] Go to the ***Insert*** tab and click ***Buttons*** → ***Navigator*** → ***Page Navigator***.

Step 2. [Format] Select the new button. In the ***Format*** pane, turn on ***Action*** and set the type to ***Page navigation***.

9.4.2 Custom Tooltips

Tooltips appear when a user hovers over a visual. You can create custom tooltip pages to show extra data without cluttering the main canvas.

Create a Custom Tooltip

Step 1. [Report View] Create a new page. In the ***Format*** pane under ***Canvas settings***, set the ***Type*** to ***Tooltip***.

Step 2. [Visualizations] Drag a ***Card*** with ***[Gross Profit]*** and a ***Card*** with ***[Total Orders]*** onto the page.

Step 3. [Report View] Go back to the Executive page. Select the Line Chart. In the ***Format*** pane, go to ***Properties*** → ***Tooltips***, and select your new tooltip page.

What you have just built. A complete, multi-page, interactive dashboard. It features a clean executive summary, a deep-dive operational page, navigation buttons, and custom tooltips. You have moved beyond simply writing formulas—you have designed a user experience.

9.5 Practice: Apply What You Have Learned

Practice

1. Change the theme of your report. Go to ***View*** → ***Themes*** and choose a professional theme (like "Executive" or "Frontier"). Notice how all the colours update automatically.
2. Add a ***Bookmark*** to the Executive page that filters the page to only show "Equipment" sales, and link it to a button.
3. Explain why it is better to have two separate pages (Executive and Operational) instead of putting all the visuals on one single page.

Solutions

1. The theme updates the default colour palettes, fonts, and backgrounds across every visual, ensuring a consistent, professional look without formatting each visual individually.
2. Insert → Buttons → Blank. Format the button's Action to trigger the Bookmark.
3. A single page with too many visuals becomes cluttered and confusing. Executive users want high-level KPIs, while operational users need granular tables. Separating them by page, linked by drillthrough or navigation buttons, keeps the dashboard focused and fast.

Part 10

Enterprise Practice: Deployment and Governance

By the end of this part you will be able to

- Describe a development, test, and production pipeline.
- Explain why version control matters for Power BI projects.
- Explain certification, promotion, and sensitivity labels.

Once a report is used by an organisation, it stops being a personal file. It becomes a shared asset, and it needs the same discipline as a software product. This part introduces the practices that professional teams use to keep reports reliable.

10.1 Deployment pipelines

A deployment pipeline moves content through three stages: Development, Test, and Production. Each stage has its own workspace and its own data source settings. A test report can therefore point at test data, while the production report points at live data. Use one pipeline for each report, rather than one for each person.

10.2 Power BI project files and version control

A **PBIP** project stores a report as a set of text files. Text files work with version-control systems such as Git. The team can then review changes, compare versions, and roll back a broken edit. A single `.pbix` file is binary, so Git cannot show what changed inside it.

⚠ Watch out

The most common failure in a team without version control is an overwritten file. Two people publish different versions of the same `.pbix`, and one set of changes disappears. Keep the work in a version-controlled folder, and agree in advance on who is allowed to publish.

10.3 Certification and labels

An owner marks a dataset as **promoted** when it is ready for others to use. A designated certifier reviews the dataset and marks it **certified**. Sensitivity labels classify content, for example as Confidential, Internal, or Public, and they can restrict sharing and export.

Practice

1. Describe the three pipeline stages, and say what changes between them.
2. Explain, in two or three sentences, why a `.pbix` file is hard to review in Git.
3. Give one reason why a dataset might carry a Confidential label.

Solutions

1. Development uses the author's own data. Test uses realistic data to validate the work. Production serves live data to consumers. The data source settings change between the stages.
2. A `.pbix` is a compressed binary file. Git stores changes as differences between text lines, so it cannot show a meaningful difference inside a binary file.
3. The dataset may contain personal or financial information that must not leave the organisation.

Part 11

NGO and Monitoring & Evaluation Patterns

By the end of this part you will be able to

- Model indicators, targets, and actuals around a shared indicator dimension.
- Build reports disaggregated by sex, age group, and location.
- Clean exported survey data from tools such as KoboToolbox and CommCare.
- Calculate achievement against donor targets.

Monitoring and evaluation work has three recurring needs. Donors track indicators against agreed targets. They want results broken down by sex, age, and place. And the survey data that feeds the report keeps changing as forms are edited in the field. This part shows you how to handle all three with the patterns you have already learned.

11.1 Indicators, targets, and actuals

Resist the temptation to link the Targets table and the Actuals table directly on an indicator name written as text. Names get spelled differently, and the links quietly break. Instead, create an **Indicators** dimension with one row per indicator, and relate both fact tables to it. This is the same star pattern you learned in Part 4.

Build the indicator model

Step 1. [Power Query] Click *Home* → *Enter Data*, and create an **Indicators** table with the columns `IndicatorID`, `IndicatorName`, and `Unit`.

Step 2. [Power Query] Create a **Targets** table with the columns `IndicatorID`, `Year`, and `TargetValue`.

Step 3. [Power Query] Create an **Actuals** table with the columns `IndicatorID`, `ReportDate`, `Sex`, `AgeGroup`, `District`, and `Value`.

Step 4. [Model View] Relate **Indicators[IndicatorID]** to **Targets[IndicatorID]**, and then to **Actuals[IndicatorID]**. Both are many to one, with a single direction.

Step 5. [Model View] Relate the **Date** table to **Actuals[ReportDate]**, exactly as you did in Part 4.

```
Total Actual   = SUM(Actuals[Value])
Total Target   = SUM(Targets[TargetValue])
Achievement %  = DIVIDE([Total Actual], [Total Target])
Gap to Target  = [Total Target] - [Total Actual]
```

Why this matters: A shared dimension means one filter, such as an indicator or a year, applies to both the targets and the actuals. Linking on text names breaks as soon as someone types an indicator differently, and the error is very hard to spot.

Watch out

Achievement can rise above 100% when targets are exceeded. Do not cap it silently to make the chart look tidy. Show the true value, and explain it in a note on the report.

11.2 Disaggregation

Donors almost always require results broken down by sex and age group. Use the age bands written in the donor's proposal. Store them as a column in the collected data, or in a small **AgeGroups** table. Keep the bands exactly as the funder defined them, so every report matches the funder's categories.

Watch out

Changing the age bands halfway through a project breaks comparisons over time. Agree the bands before collecting any data, and write them down.

11.3 Cleaning KoboToolbox and CommCare exports

Exported survey columns often carry long technical names, such as `group_q1_respondent_name`. Clean them once in Power Query, so the same steps run automatically on every new export.

Clean a survey export

Step 1. [Power Query] Import the export as a CSV or Excel file.

Step 2. [Power Query] Select the columns you do not need, right-click them, and choose ***Remove Columns***.

Step 3. [Power Query] Double-click each remaining header and rename it. For example, `group_q1_respondent_name` becomes `RespondentName`.

Step 4. [Power Query] Select the `Sex` column, then click ***Transform*** → ***Replace Values***. Replace `M` with `Male` and `F` with `Female`.

Step 5. [Power Query] Click ***Close & Apply***.

Watch out

When a form changes, the export gains new columns or renames old ones. Check the column list after every import. A step that refers to a missing column will fail, and the refresh will stop with an error. That is far better than a silent wrong total.

Practice

1. Build an Indicators dimension and two fact tables. Write a measure that shows achievement by indicator.
2. Create a matrix with Sex on the rows and Total Actual in the values. Check that the male, female, and any other categories add up to the overall total.

3. Write a rule for handling a survey export in which a question has been renamed in the form. Explain how you would detect the change.

Solutions

1. Use the Indicators table as the rows of a matrix, with `Achievement %` in the values. Each indicator then shows its own percentage.
2. The categories must add up to the overall total. If they do not, some rows have a blank or unexpected sex value. Add an "Unknown" category so the gap becomes visible.
3. Compare the list of column names with the expected list after each import. A missing column should stop the refresh with a clear message, not produce a silent wrong total.

Part 12

Exam Preparation

By the end of this part you will be able to

- Map each chapter to the skills measured by the certification exam.
- Answer practice questions and explain your reasoning.
- Work through scenario questions that resemble real business requests.

Passing the exam is not about memorising functions. It is about recognising the situation a question describes and knowing which feature solves it. The questions below are written in that style. For each one, pause, decide what you think the answer is, and only then read the key.

12.1 Mapping to the exam

Microsoft updates the skills outline from time to time, so always check the current version on the certification website before you sit the exam.

Exam area	Where this course covers it
Prepare the data	Parts 2 and 3
Model the data	Part 4
Create calculations with DAX	Part 5
Visualise and analyse	Part 6
Deploy and maintain assets (security, refresh, workspaces)	Parts 7 and 10
Performance and advanced practice	Parts 8 and 11

12.2 Practice questions

Answer each question before you read the key. Write down why you chose your answer, because the reasoning is what the examiners are really testing.

1. Which Power Query feature creates a second query that follows the first query's output, so that a change to the first flows into the second?

   a) Duplicate
   b) Reference
   c) Append
   d) Merge

2. A relationship between Sales and Date is set to filter in both directions. What should you check first?

   a) Refresh speed
   b) Ambiguous filter paths and inflated totals
   c) Blank time-intelligence results
   d) Gateway settings

3. Which join returns the customers who have never placed an order?

   a) Left outer
   b) Inner
   c) Left anti
   d) Full outer

4. Which workspace role is subject to row-level security?

   a) Admin
   b) Member
   c) Contributor
   d) Viewer

5. Which measure returns the total of the last three months, moving with the current month?

   a) `CALCULATE([Total Sales], DATESINPERIOD('Date'[Date], MAX('Date'[Date]), -3, MONTH))`
   b) `TOTALMTD([Total Sales], 'Date'[Date])`
   c) `SUM(Sales[LineAmount]) / 3`
   d) `CALCULATE([Total Sales], 'Date'[Month Number] = 3)`

6. A visual has Customers[CustomerName] on its rows. The measure is `CALCULATE([Total Sales], KEEPFILTERS(Customers[Country] = "Kenya"))`. What does the Uganda row show?

   a) The Kenya total
   b) The Uganda total
   c) Blank
   d) An error

7. The same visual uses `CALCULATE([Total Sales], Customers[Country] = "Kenya")`, without KEEPFILTERS. What does the Uganda row show?

   a) Blank
   b) The Uganda total
   c) The Kenya total
   d) Zero

8. A Type 2 customer dimension has two rows for one customer. Which measure counts each customer once?

   a) `COUNTROWS(Customers)`
   b) `DISTINCTCOUNT` on the surrogate key
   c) `DISTINCTCOUNT` on the business key

d) `COUNT(Customers[Country])`

9. Which function identifies the signed-in user for dynamic RLS in the Service?

   a) `USERNAME()`
   b) `USERPRINCIPALNAME()`
   c) `ISFILTERED()`
   d) `HASONEVALUE()`

10. Object-level security is used to:

    a) Filter rows for a role
    b) Hide a table or column from a role
    c) Encrypt data at rest
    d) Schedule a refresh

11. Which storage mode keeps no in-memory copy of the data?

    a) Import
    b) DirectQuery
    c) Dual
    d) Calculated table

12.3 Answer key with reasoning

1. **b) Reference.** A reference query starts from the first query's output, so later changes flow through. A duplicate copies the steps, and the two queries then go their separate ways.

2. **b) Ambiguous filter paths and inflated totals.** Two-way filtering gives Power BI more than one route between tables, and totals can be counted more than once.

3. **c) Left anti.** It keeps the rows from the first table that have no match in the second, which is exactly what "never ordered" means.

4. **d) Viewer.** Viewers are subject to RLS. Contributors, Members, and Admins bypass it.

5. **a) The DATESINPERIOD version.** It creates a moving three-month window based on the latest date in the current filter context.

6. **c) Blank.** KEEPFILTERS intersects the Uganda row with the Kenya filter. No row is both, so the result is blank.

7. **c) The Kenya total.** Without KEEPFILTERS, the filter replaces the country that the visual was showing. Every row therefore displays the Kenya figure.

8. **c) DISTINCTCOUNT on the business key.** The business key stays the same across every version of a customer, so counting distinct business keys counts each real customer once. The surrogate key changes with each version and would count the same customer twice.

9. **b) USERPRINCIPALNAME().** It is the reliable choice in the Service. USERNAME() behaves inconsistently between Desktop and the Service.

10. **b) Hides a table or column from a role.** Object-level security removes whole objects. RLS filters rows.

11. **b) DirectQuery.** It sends queries to the source, so nothing is held in the model's memory.

A word from your lecturer

Notice that questions 6 and 7 use the same measure with and without KEEPFILTERS, and the answers differ. That is deliberate. Examiners love to test whether you understand what a single word changes in a formula. Read every option carefully, and predict the result before you look at the choices.

12.4 Scenario questions

The exam is largely scenario-based, so practise with stories like these. Read the situation, identify the business need, and then choose the feature that solves it.

Practice

1. A company has ten million rows of sales data. It refreshes once a day, and the report only needs yesterday's totals. Should it use Import or DirectQuery? Give two reasons.
2. A regional manager sees a Uganda total that includes Kenyan customers. Which table would you check first, and what would you look for?
3. A report must show the same product list to every user, but each country manager may see only their own customers. Which security feature would you use, and on which table would you place it?

Solutions

1. Import is usually the better choice. Yesterday's totals from a daily refresh are small to refresh and give fast visuals. DirectQuery would send a query for every visual each time someone opens the report, and it would load the source system. DirectQuery would only make sense if the data had to be live throughout the day.
2. Check the relationship between Sales and Customers. Look for a bi-directional filter, a mismatched key, or a filter applied to the wrong table.
3. Row-level security, placed on the Customers table, either static or dynamic. Products stays unrestricted, and the customer rows are limited to each manager's countries.

Part 13

Capstone Project

By the end of this part you will be able to

- Repeat the full workflow on a new dataset without step-by-step instructions.
- Deliver a model, measures, pages, security, and a refresh plan.

This is where you show what you have learned. You will receive no step-by-step instructions, only a brief and a marking rubric. It is the closest thing this course has to a real project at work, and it is the best preparation for the exam's practical thinking.

13.1 Brief

Build a report for a fictional organisation that sells equipment across three countries. The regional director wants answers to four questions. How are sales trending? Which products drive the margin? Which customers have become inactive? And who is allowed to see what?

13.2 Deliverables

1. A cleaned data model with a fact table, at least two dimensions, a marked Date table, and a grain that you have written down.
2. At least ten DAX measures, including one time-intelligence measure, one ranking, and one measure that uses `KEEPFILTERS`.
3. Three report pages: an Overview, a Products page, and a Customer Detail page reached through drillthrough.
4. Static and dynamic row-level security, tested with a Viewer account.
5. A short refresh plan that names the source and the refresh frequency, and explains why you chose them.

13.3 Marking rubric

Area	What earns full marks	Points
Data preparation	Typed columns, a parameterised path, and clearly documented steps	15
Model	A star schema, correct cardinality, and a marked Date table	20
DAX	Correct results, sensible use of variables, and correct handling of context	25
Visuals	Pages that answer questions, working interactions, and a readable design	20
Security and refresh	RLS tested with a Viewer, and a realistic refresh plan	20
	Total	**100**

Part A

Data File Reference

The course uses four CSV files. Create them yourself, or paste in the exact contents supplied with the course. Column names must match exactly, because the Power Query steps refer to columns by name.

File	Expected columns
`Sales.csv`	OrderID, OrderDate, CustomerKey, ProductKey, Quantity, UnitPrice
`Sales_2023_Extension.csv`	The same six columns as Sales.csv
`Customers.csv`	CustomerKey, CustomerName, Segment, Country
`Products.csv`	ProductKey, ProductName, Category, UnitCost

Watch out

Do not open the CSV files in Excel before loading them. Excel can change the encoding and the date formats, and Power Query will then read values that differ from the originals.

Part B

Cheat Sheets

Power Query (M) quick reference

Task	Where or how
Change a column's type	Click the type icon in the column header
Add a calculated column	Add Column → Custom Column
Stack tables on top of each other	Home → Append Queries
Join tables side by side	Home → Merge Queries, then choose the join kind
Keep only rows with no match	Merge Queries → Join Kind: Left Anti
Turn columns into rows	Transform → Unpivot Columns
Summarise rows	Home → Group By
Find and replace text	Transform → Replace Values
Add a time offset	`[Col] + #duration(0, 2, 0, 0)`
Handle an error in one expression	`try [Col] otherwise 0`

DAX quick reference

Task	Function
Total, average, minimum, or maximum of a column	`SUM`, `AVERAGE`, `MIN`, `MAX`
Count distinct keys	`DISTINCTCOUNT`
Change a filter	`CALCULATE(measure, filter)`
Intersect with the existing filter	`KEEPFILTERS(filter)`
Remove filters	`ALL`, `REMOVEFILTERS`
Row-by-row calculation	`SUMX`, `AVERAGEX`, `MAXX`
Pull a value from the "one" side	`RELATED`
Safe division	`DIVIDE(a, b)`
Year to date	`TOTALYTD(measure, 'Date'[Date])`
Same period last year	`SAMEPERIODLASTYEAR('Date'[Date])`
Rank items in a table	`RANKX(ALL(Table[Column]), [Measure])`
Top N items	`TOPN(N, Table, [Measure], DESC)`
Store a value under a name	`VAR name = ...` followed by `RETURN`
Identify the current user (RLS)	`USERPRINCIPALNAME()`

Dashboard Creation quick reference

Task	Where or how
Change the report colours	View → Themes → Choose a theme
Make a visual fill the screen	Format Visual → General → Properties → Size
Filter only one visual	Format → Edit interactions
Jump to a detail page	Drag a field into the ***Drill through*** well on a new page
Save a specific view	View → Bookmarks → Add
Create a hover-over tooltip	Create a page → Format Page → Canvas Settings → Type: Tooltip
Add page navigation	Insert → Buttons → Navigator → Page navigator
Hide a visual from the reader	Format → General → Properties → Hide
Lock a visual in place	Format → General → Properties → Lock aspect

Part C

Glossary

Calculated column A column computed row by row at refresh time and stored in the model.

Cardinality The number of distinct values in a column, or the type of a relationship (one-to-many, many-to-many).

Context The row or filter conditions under which a DAX formula is evaluated.

Fact table A table of events or numbers, such as sales.

Filter context The set of rows currently visible, set by slicers, axes, and filters.

Grain What one row in a table represents.

Measure A formula evaluated at query time, for the filters currently applied.

Query folding Translating Power Query steps into the source system's language, so the source does the work.

Row context The current row during a calculation that works row by row.

Star schema A fact table surrounded by dimension tables.

Disclaimer. Power BI is updated regularly, so menu names, the ribbon layout, and button positions may differ slightly in newer versions. The concepts, the DAX, and the Power Query code in this course remain the same unless noted otherwise. When a screen does not match the course, check Microsoft's current documentation. The diagrams are schematic drawings, not screenshots.

Part D

About the Author and Disclaimer

Jean Baptiste Habanabakize is a Data Analytics Specialist and holds a Master's degree in Big Data Analytics (Data Science), 2026.**Jean Baptiste Habanabakize** is a Data Analytics Specialist and holds a Master's degree in Big Data Analytics (Data Science). He brings a unique perspective to data work, having spent twelve years as a science teacher before transitioning into full-time analytics and MEL consultancy. He has since applied his expertise in Python, R, and Power BI across the education, public health, and agriculture sectors, working with organizations such as EDUSCA, the Rwanda Biomedical Centre, and One Acre Fund.